

Mem0 v3 算法拆解 (2026-04)

Additive Memory + Multi-Signal Retrieval

对照 Mem0 2025 论文 (Mem0 base + Mem0^g) 的 Step / Input / Output / Algorithm 格式, 拆解当前代码库实现。

- 落地版本: commit `a488e190` — *feat(oss): port v3 pipeline with hybrid search, entity extraction, and additive scoring* (PR #4805)
- 核心实现: `mem0/memory/main.py`、`mem0/configs/prompts.py`、`mem0/utils/scoring.py`
- 迁移文档: `docs/migration/oss-v2-to-v3.mdx`

0. Notation Setup

Step	Input	Output	Algorithm
Notation Setup	Database	M: 记忆库 (主向量集合) $m_i = (\text{id}_{\text{uuid}}, \text{text}, s_{m_i}, \text{payload})$ s_{m_i}: 记忆文本的语义向量 payload 含 $(\text{hash}_{\text{md5}}, \text{text_lemmatized}, \text{attributed_to}, t_{\text{created}}, t_{\text{updated}}, \text{filters})$ N: 实体库 (独立向量集合 <code>{collection}_entities</code>) $e_j = (\text{text}, \text{type}, s_{e_j}, L_j)$ L_j: <code>linked_memory_ids</code>, 该实体指向的记忆 id 集合 H: SQLite 消息表, 按 σ 滚动保留最近 10 条 σ: session scope, 由 $(\text{user_id}, \text{agent_id}, \text{run_id})$ 转义拼接	无标记图: $\mathcal{D} = (M, N, H)$ 与论文对比: E (标记边) 和 L (节点语义类型的图角色) 不存在。N 是实体→记忆的倒排索引, 实体之间无边, 故无关系推理、无 invalid 标记、无时序失效

1. ADD 路径

`Memory._add_to_vector_store(infer=True)` — `main.py:916-1206`

Step	Input	Output	Algorithm
Phase 0 Context Gathering	1. 新消息 m_t (可为单条或整段多轮) 2. filters	σ : H_k (最近 $k=10$ 条历史消息); T (扁平化文本)	$\sigma = \text{buildSessionScope(filters)}$ $H_k = \text{db.getLastMessages}(\sigma, 10)$ $T = \text{parseMessages}(m_t)$

Step	Input	Output	Algorithm
Phase 1 Existing Retrieval	T , filters	$M_{\text{ctx}} = \{("0", \text{text}_0), \dots, ("9", \text{text}_9)\}$ $\text{uuidMap} : "i" \mapsto \text{uuid}_i$	$q = \text{embed}(T, \text{"search"})$ $M_{\text{ctx}} = \text{vectorStore.search}(q, \text{top_k}=10, \text{filters})$ $\text{UUID} \rightarrow \text{序号整数映射}$ (anti-hallucination: LLM 只看 "0", "1" ,...)
Phase 2 Extraction (唯一一次 LLM 调用)	$P = (S, H_k, R, M_{\text{ctx}}, T, d_{\text{obs}}, d_{\text{cur}}, I_{\text{custom}})$	$F = \{\omega_1, \dots, \omega_n\}$ $\omega_i = (\text{id}_i, \text{text}_i, \text{attributed_to}_i, \Lambda_i)$ $\Lambda_i \subseteq \text{linked_memory_ids}$ (UUID)	$\Phi_{\text{ADD}}(P) = F$, system prompt = <code>ADDITIVE_EXTRACTION_PROMPT</code> (agent-scoped 时追加 <code>AGENT_CONTEXT_SUFFIX</code>) 输出算子只有 ADD — 无 UPDATE / DELETE / NOOP 分支 约束: 15–80 词、self-contained、保留专有名词与数量、显式编码转变关系 (从 X 变为 Y) 抽取源含 user 与 assistant 双方消息
Phase 3 Batch Embed	$\{\text{text}_i\}_{i=1}^n$	$\{s_{\omega_i}\}$	$\text{embedBatch}(\cdot, \text{"add"})$, 失败降级为逐条 <code>embed()</code>
Phase 4–5 CPU 处理 + Hash 去重	F , M_{ctx} 的 hash 集合	$\text{records} = \{(\text{uuid}_i, \text{text}_i, s_{\omega_i}, \text{payload}_i)\}$	$h_i = \text{MD5}(\text{text}_i)$ 丢弃条件: $h_i \in \text{hashes}(M_{\text{ctx}})$ (跨批) 或 $h_i \in \text{seen}$ (批内) $\text{text_lemmatized}_i = \text{lemmatizeForBM25}(\text{text}_i)$ (spaCy) $t_{\text{updated}} \leftarrow t_{\text{created}}$
Phase 6 Batch Persist	records	$M_t = M_{t-1} \cup \text{records}$	$\text{vectorStore.insert}(\vec{v}, \vec{\text{id}}, \vec{p})$ 单次批量; 失败降级逐条 $\text{db.batchAddHistory}$, 事件恒为 <code>"ADD"</code> , <code>old_memory = None</code>
Phase 7 Entity Linking	$\{\text{text}_i\}$, filters	N_i (新增/更新的实体节点)	7a $\mathcal{E} = \text{extractEntitiesBatch}(\{\text{text}_i\})$ (spaCy NER + 规则) 按 $\text{norm}(e) = \text{lower}(\text{strip}(e))$ 全局归并 $\rightarrow e \mapsto (\text{type}, \text{text}, \{\text{mem_id}\})$ 7b 唯一实体单次 <code>embedBatch</code> ; 长度不匹配则 pad/truncate 而非丢弃 7c <code>entityStore.searchBatch(top_k=1)</code> 7d $\text{match} = \text{exactMatch} \vee (\text{semanticMatch if score} \geq 0.95)$ 命中: $L_j \leftarrow L_j \cup \{\text{mem_ids}\}$ (只并集, 不删) 未命中: 新建节点 7e 新实体单次批量 insert 整个 Phase 7 包在 <code>try/except</code> 里 — 失败只 warn , 不影响记忆写入
Phase 8 Persist Messages	m_t, σ	返回 $\{(\text{id}, \text{memory}, \text{event}=\text{"ADD"})\}$	$\text{db.saveMessages}(m_t, \sigma)$, 并驱逐 σ 下第 10 条之后的旧消息

LLM 调用数: **1** (恒定)。论文为 **1 + n** (每条候选 fact 一次 UPDATE 决策)。这是延迟减半的直接来源。

2. SEARCH 路径

Memory._search_vector_store() — main.py:1628-1731

Step	Input	Output	Algorithm
Step 1 Query 预处理	query q	q_{lem} , E_q	$q_{\text{lem}} = \text{lemmatizeForBM25}(q)$ $E_q = \text{extractEntities}(q)$
Step 2-3 语义召回	q , filters	候选池 C	$\text{internalLimit} = \max(4 \cdot \text{limit}, 60)$ (over-fetch) $C = \text{vectorStore.search}(\text{embed}(q), \text{internalLimit}, \text{filters})$, 过滤已过期 payload
Step 4-5 BM25 信号	q_{lem}	$B : \text{id} \mapsto [0, 1]$	$R_{\text{kw}} = \text{vectorStore.keywordSearch}(q_{\text{lem}}, \text{internalLimit})$ 不支持的 store 返回 <code>None</code> → 信号静默关闭 $(\mu, \kappa) = \text{getBM25Params}(q_{\text{lem}})$, 按词数分段: $\leq 3 \rightarrow (5.0, 0.7)$, $\leq 6 \rightarrow (7.0, 0.6)$, $\leq 9 \rightarrow (9.0, 0.5)$, $\leq 15 \rightarrow (10.0, 0.5)$, else $(12.0, 0.5)$ $B_{\text{id}} = \frac{1}{1 + e^{-\kappa(\text{raw} - \mu)}}$
Step 6 Entity Boost	E_q , filters	$\beta : \text{id} \mapsto [0, 0.5]$	归一化去重后取前 8 个实体, <code>embedBatch</code> 后 4 线程并发查 N (<code>top_k=500</code>) 对每个匹配实体 e_j , 设 $\text{sim}_j = \text{score}(e_j)$, 门限 $\text{sim}_j \geq 0.5$: $w_j = \frac{1}{1 + 0.001 (L_j - 1)^2}$ (枢纽实体衰减) $\text{boost}_j = \text{sim}_j \cdot W_{\text{ent}} \cdot w_j$, $W_{\text{ent}} = 0.5$ $\forall \text{id} \in L_j : \beta_{\text{id}} \leftarrow \max(\beta_{\text{id}}, \text{boost}_j)$ (取 max, 不累加)
Step 7-8 加性融合与排序	C , B , β , θ , k	Top- k 结果	$D = 1.0 + \mathbb{1}[B \neq \emptyset] \cdot 1.0 + \mathbb{1}[\beta \neq \emptyset] \cdot 0.5 \in \{1.0, 1.5, 2.0, 2.5\}$ 门限先于融合: 若 $\text{sem}_i < \theta$ (默认 $\theta = 0.1$) 则直接丢弃 — BM25 与实体分无法救回低语义分候选 $\text{score}_i = \min \left(\frac{\text{sem}_i + B_i + \beta_i}{D}, 1.0 \right)$ 降序排序取前 k (默认 $k = 20$)。 D 自适应保证不支持 BM25 的 store 不被系统性压分
Step 9 结果格式化	scored results	<code>MemoryItem[]</code>	提升 payload 中 (<code>user_id, agent_id, run_id, actor_id, role, attributed_to, expiration_date</code>) 到顶层, 其余归入 <code>metadata</code> ; <code>explain=True</code> 时附 <code>score_details</code>

3. 与论文的算子级差异

论文算子 (2025)	v3 现状 (2026-04)
$\Phi(P) = F$ (提取)	保留, 但 P 的构成与输出 schema 变了
$K_{ij} = g(\omega_i, m_j)$ (写入期相似度)	删除。写入期不再算候选↔存量的相似度, 只算 MD5 相等
Top- $s=10$ 选择 + 四算子决策	删除。 M_{ctx} 的 top-10 仅作为 prompt 上下文供软去重和链接, 不驱动任何写操作
ADD / UPDATE / DELETE / NOOP	只剩 ADD 。NOOP 退化为 prompt 层跳过 + hash 丢弃; UPDATE 退化为 Δ_i 链接; DELETE 无对应物
$g(T) = V, \quad h(V) = E$ (图抽取)	g 由 LLM 降为 spaCy NER ; h 无对应实现
Conflict Detection + Update Resolver	无对应实现。矛盾记忆共存于 M
Summary Generator $\rightarrow S$	参数槽保留, OSS 未接线 (见第 5 节)

3.1 Mem0^g 的替代物对比

Mem0^g (论文)	Entity Store (现在)
$G = (V, E, L)$, 有标记边	只有节点, 没有边
LLM 生成 relationship	spaCy NER + 规则抽实体 (<code>entity_extraction.py</code> , 720 行)
Update Resolver 标记边 invalid	无 — 实体只维护 <code>linked_memory_ids</code> 倒排表
用于图遍历检索	用于检索期加分

准确说, N 是实体倒排索引, 不是知识图谱。关系推理能力是净损失, 换来的是无外部图数据库依赖。

3.2 死代码

以下仍在仓库中但已无调用点:

- `DEFAULT_UPDATE_MEMORY_PROMPT` — `prompts.py:176`
- `get_update_memory_messages()` — `prompts.py:406`
- `FACT_RETRIEVAL_PROMPT` — 仅被 `get_fact_retrieval_messages_legacy()` 引用

3.3 过期文档

`mem0/CLAUDE.md` 仍写有 `Graph stores | 4 | Neo4j, Memgraph, Kuzu, Apache AGE` 及整节 "Graph memory", 但 `mem0/graphs/` 目录已删除, `graph_store / enable_graph` 配置已移除。文档与代码不一致。

4. 设计哲学的转变

论文的核心假设是「记忆库应保持一致」——所以需要 Update Resolver 消解冲突。

v3 的核心假设是「记忆只累积，不覆盖」(README: *Memories accumulate; nothing is overwritten*)。矛盾信息共存，交由检索期排序解决。

这一转变带来两个连锁设计：

1. 反原子化。既然不能改写旧记忆，新记忆必须自带「从什么变成什么」的转变信息，否则系统丢失变更语义。[prompts.py:607-620](#) 明确要求 *Contextually Rich, Not Atomic*：

Bad: "User prefers oat milk lattes" Good: "User switched from almond milk to oat milk lattes after developing an almond sensitivity"

2. 检索期承担全部消歧负担。论文的检索只是 $K_{ij} + \text{top-}s$ ；v3 变成三信号加性融合 + 时序排序，因为写入期不再做任何冲突判断。

5. OSS 未接线的输入槽

`generate_additive_extraction_prompt()` 的签名支持 7 个输入槽，但 [main.py:948-953](#) 只传了 4 个：

槽位	OSS 实际值	后果
<code>summary</code> S	<code>None</code>	论文 $P = (S, \dots)$ 的 S 项恒为空，Summary Generator 在 Platform 侧
<code>recently_extracted_memories</code> R	<code>None</code>	会话内跨批去重参考恒为空
<code>timestamp</code> $\rightarrow d_{\text{obs}}$	<code>None</code>	$d_{\text{obs}} = d_{\text{cur}} = \text{today}$
<code>use_input_language</code>	<code>False</code>	多语言抽取默认关闭

第三行影响最大：prompt 中大段「用 Observation Date 而非 Current Date 锚定相对时间」的规则，在 OSS 中因两者恒等而退化为「一律锚定到今天」。历史对话回填时，`"last week"` 会被锚到导入当天而非对话真实发生时间。

这与 [notices.py:45-49](#) 声明的 `timestamp` / `reference_date` / `decay` 参数 OSS 不支持是同一件事。

6. 破坏性 API 变更

来自 [docs/migration/oss-v2-to-v3.mdx](#)：

项	旧	新
<code>search()</code> / <code>get_all()</code> 的 entity id	顶层 kwarg	必须放进 <code>filters</code> ，顶层传抛 <code>ValueError</code>
<code>top_k</code> 默认	100	20
<code>threshold</code> 默认	<code>None</code> （不过滤）	0.1，且必须在 <code>[0, 1]</code>
<code>rerank</code> 默认	<code>True</code>	<code>False</code>
<code>add()</code> 返回事件	ADD / UPDATE / DELETE	只有 ADD
<code>custom_fact_extraction_prompt</code>	—	→ <code>custom_instructions</code>
<code>custom_update_memory_prompt</code>	—	废弃
图记忆	<code>enable_graph</code> + <code>graph_store</code>	移除，转为 Platform 内置 always-on
Qdrant client	<code>>=1.9.1</code>	<code>>=1.12.0</code>

新增依赖：spaCy。

```
pip install --upgrade "mem0ai[nlp]"
python -m spacy download en_core_web_sm
```

不安装则 BM25 与实体信号静默降级（*D* 自适应回落到 1.0，退化为纯语义检索）。

7. 基准分数的适用范围

README 报告：

Benchmark	Old	New	Tokens	Latency p50
LoCoMo	71.4	92.5	7.0K	0.88s
LongMemEval	67.8	94.4	6.8K	1.09s
BEAM (1M)	—	64.1	6.7K	1.00s
BEAM (10M)	—	48.6	6.9K	1.05s

README 同段声明：

Scores reflect Mem0's managed platform, which includes proprietary optimizations not available in the open-source SDK; open-source users should expect directionally similar gains but not identical numbers.

结合第 5 节，OSS 与 Platform 的实际能力差为：

能力	OSS	Platform
ADD-only 单次抽取	✓	✓
三信号混合检索	✓	✓
实体倒排 + boost	✓	✓
图记忆	✗	✓（内置 always-on）
Summary Generator	✗	✓
Temporal Reasoning	✗	✓
Decay	✗	✓

论文描述的那个双轨系统（Mem0 + Mem0^g）作为一个开源整体已不存在。